

# Table of Contents

|                                                    |   |
|----------------------------------------------------|---|
| Supplementary Requirements Documentation .....     | 2 |
| Interactive House .....                            | 2 |
| Revision History.....                              | 2 |
| Supplementary Requirements List .....              | 2 |
| Supplementary Requirements Descriptions .....      | 3 |
| S1. Data persistency & consistency .....           | 3 |
| S2. Database performance & latency .....           | 3 |
| S3. Scalability of data storage .....              | 3 |
| S4. Data security & access control .....           | 3 |
| S5. Backup & recovery strategy .....               | 4 |
| S6. Availability & fault tolerance .....           | 4 |
| S7. Database technology constraints .....          | 4 |
| S8. Deployment & Architecture Considerations ..... | 4 |

# Supplementary Requirements Documentation

## Interactive House

### Revision History

| Date     | Version | Description                                                                                                                                                                    | Author                                        |
|----------|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| 06/02/26 | 1.0     | Initial supplementary requirements focusing on database POV                                                                                                                    | Katerina Arvay, Dario Ostojic, Andre Sandblom |
| 01/03/26 | 1.1     | Update of supplementary requirements to reflect the adoption of Firebase Firestore Database                                                                                    | Katerina Arvay, Dario Ostojic, Andre Sandblom |
| 17/03/26 | 1.2     | Extended supplementary requirements to include deployment considerations and hybrid architecture (Firebase Firestore + gateway)                                                | Katerina Arvay, Dario Ostojic, Andre Sandblom |
| 23/3/26  | 1.3     | Revised wording to include a dual database structure                                                                                                                           | Katerina Arvay, Dario Ostojic, Andre Sandblom |
| 08/05/26 | 1.4     | S5 Not Implemented (Supabase is stub code); S4 Partial (SG1 Firestore rules wide open); S7 Partial (only primary DB operational); S8 elevated to Essential; priorities updated | Katerina Arvay, Dario Ostojic, Andre Sandblom |

### Supplementary Requirements List

| Supplementary Requirement Name               | Priority  |
|----------------------------------------------|-----------|
| S1. Data persistence & consistency           | Essential |
| S2. Database performance & latency           | Desirable |
| S3. Scalability of data storage              | Desirable |
| S4. Data security & access control           | Essential |
| S5. Backup & recovery strategy               | Desirable |
| S6. Availability & fault tolerance           | Desirable |
| S7. Database technology constraints          | Desirable |
| S8. Deployment & Architecture Considerations | Essential |

# Supplementary Requirements Descriptions

## S1. Data persistency & consistency

All smart home data, including user accounts, smart devices, device states, schedules, and historical logs, shall be persistently stored within the system's data storage solutions.

The system shall ensure logical data consistency such that stored device states accurately reflect the real-world state of devices after successful operations.

Where multiple updates occur concurrently, appropriate mechanisms shall be used to prevent partial or inconsistent updates.

The system shall ensure consistency between primary and secondary data storage to avoid conflicting system states.

Real-time synchronization shall ensure that all connected clients receive consistent and up-to-date device information.

## S2. Database performance & latency

Database operations related to critical functionality, such as retrieving device status or updating device states, shall respond within a maximum of 2 seconds under normal operating conditions.

The system shall utilize Firebase's real-time data synchronization capabilities to ensure low-latency updates between clients and the database.

The database shall efficiently support frequent read and write operations generated by multiple users and smart devices simultaneously.

## S3. Scalability of data storage

The system shall support scalability to handle an increasing number of users, smart devices, automation rules, and sensor data over time.

The data storage solution shall support horizontal scaling and efficient handling of concurrent connections as the system grows.

## S4. Data security & access control

The database shall enforce strict access control to ensure that users can only access data related to their own smart home environment.

Authentication and authorization mechanisms shall ensure that only authorized users and system components can access data.

Access control policies shall restrict read and write operations appropriately.

Sensitive information shall not be stored in plain text and shall follow secure handling practices.

Database access shall be limited to authorized backend components only. Note: the SG1 Firebase project's Firestore security rules are currently a temporary development setting allowing all reads and writes until October 2026. Proper role-based rules must be deployed to the SG1 project before production use.

## **S5. Backup & recovery strategy**

The system shall implement a backup and recovery strategy using a secondary data storage solution.

Critical system data shall be replicated or backed up to ensure durability in case of failure of the primary database.

The system shall support recovery to a recent consistent state in the event of data loss, corruption, or system failure.

Backup mechanisms shall minimize data loss and ensure continuity of system operation. Note: this requirement is not currently implemented. The Supabase folder contains only placeholder stub code. The system relies entirely on Firebase Firestore with no backup or recovery path. This remains as future work.

## **S6. Availability & fault tolerance**

The system shall rely on Firebase's distributed cloud infrastructure to ensure high availability.

Firebase's built-in replication and redundancy mechanisms shall minimize downtime and reduce the risk of data loss due to hardware or software failures.

The system should continue operating under partial network disruptions, with automatic synchronization when connectivity is restored.

The system shall continue operating in a degraded mode if one data storage solution becomes unavailable, where possible.

## **S7. Database technology constraints**

The system shall use a combination of a primary NoSQL database and a secondary relational database to support data storage requirements.

The primary database shall support real-time synchronization of device states between clients and the backend.

The secondary database shall provide structured storage and support backup and recovery functionality.

The data model shall be designed to accommodate both NoSQL and relational storage paradigms. Note: only the primary NoSQL database (Firebase Firestore) is currently operational. The secondary relational database (Supabase) has not been implemented.

## **S8. Deployment & Architecture Considerations**

The system is deployed using a hybrid architecture. The web application is deployed as a cloud-hosted web service (Render), while the backend communication layer is handled through a locally hosted gateway service.

The gateway exposes a REST API and is responsible for forwarding commands to the smart house hardware and updating the resulting state in Firebase Firestore.

To enable external access to the locally hosted gateway, a tunneling solution (e.g. Cloudflare Tunnel) may be used to provide a publicly accessible API endpoint without requiring full backend deployment to the cloud.

This approach reduces backend complexity while preserving real-time synchronization and ensuring that device state reflects actual hardware behavior.

The architecture shall support integration with both primary and secondary data storage solutions to ensure reliability and data durability.